

Aprendizaje Automático

Resumen de conceptos y fórmulas — Primer parcial

FCEyN – UBA • 1c 2026 • Guías 1–9 (sin clustering)

Cómo leerlo: **azul** = fórmula/definición a saber de memoria; **rojo** = trampa típica de V/F; **verde** = ejemplo resuelto o número de referencia.

Índice general

1 Probabilidad y estadística	2	5.2 Generativos vs. discriminativos	7
1.1 Reglas básicas	2	5.3 KNN	7
1.2 Media, desvío y normal	2	5.4 LDA y QDA	7
1.3 Máxima verosimilitud (MLE)	2	5.5 Naive Bayes	8
2 Introducción: tipos de problemas, hipótesis, sesgo inductivo	3	5.6 Regresión logística	8
2.1 Tipos de problema	3	5.7 SVM	8
2.2 Espacio de instancias y de hipótesis	3	5.8 Fronteras de decisión típicas	9
2.3 Aprendizaje de conceptos (Mitchell cap. 1–2)	3	6 Métricas	9
2.4 Entropía e información	3	6.1 Regresión	9
3 Árboles de decisión	4	6.2 Matriz de confusión y métricas de umbral fijo	9
3.1 Medidas de impureza y ganancia	4	6.3 Umbral, ROC y PR	10
3.2 Algoritmo (ID3 / CART)	4	6.4 Multiclase y costos	10
3.3 Importancia de atributos	5	7 Sesgo y varianza	11
3.4 Propiedades y fronteras	5	8 Ensamblados	11
4 Selección de modelos, validación cruzada y Control	5	8.1 Bagging	11
4.1 Esquema Desarrollo / Control	5	8.2 Random Forest (Breiman 2001)	12
4.2 Optimismo y fugas de información (leakage)	6	8.3 Boosting	12
4.3 Métrica global vs. promedio de folds	6	9 Ingeniería de atributos	12
5 Clasificadores	7	9.1 Valores faltantes	12
5.1 Clasificador óptimo de Bayes	7	9.2 Categóricas	13
		9.3 Nuevos atributos (ejemplos para KNN)	13
		10 Chuleta final	13

1.1 Reglas básicas

Definiciones

$$P(A | B) = \frac{P(A \cap B)}{P(B)} \quad A \perp B \iff P(A \cap B) = P(A)P(B)$$

$$\text{Prob. total: } P(B) = \sum_{i=1}^n P(B | A_i) P(A_i) \quad (\{A_i\} \text{ partición, } P(A_i) > 0)$$

$$\text{Bayes: } P(A | B) = \frac{P(B | A)P(A)}{P(B)} = \frac{P(B | A)P(A)}{\sum_j P(B | A_j)P(A_j)}$$

- **Demostración de prob. total:** $B = \bigcup_i (B \cap A_i)$ unión disjunta $\Rightarrow P(B) = \sum_i P(B \cap A_i) = \sum_i P(B | A_i)P(A_i)$.
- **Demostración de Bayes:** $P(A \cap B) = P(A | B)P(B) = P(B | A)P(A)$; despejar.
- **Independencia de a pares $\neq$ mutua:** se necesita además $P(A \cap B \cap C) = P(A)P(B)P(C)$. Dos monedas: $A=1$ ra cara, $B=2$ da cara, C =iguales. $P(\cdot) = \frac{1}{2}$ c/u, cada intersección de a dos = $\frac{1}{4}$ ✓, pero $P(A \cap B \cap C) = \frac{1}{4} \neq \frac{1}{8}$ (conociendo dos, el tercero queda determinado).
- Terminología de Bayes: $P(A)$ prior, $P(B | A)$ verosimilitud, $P(A | B)$ posterior, $P(B)$ evidencia.

1.2 Media, desvío y normal

Normal

$$X \sim \mathcal{N}(\mu, \sigma^2) : f(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right), \quad z = \frac{x-\mu}{\sigma}$$

- **Media** $\mu = \mathbb{E}[X]$: centro de masa. **Desvío** $\sigma = \sqrt{\mathbb{E}[(X-\mu)^2]}$: dispersión típica alrededor de la media, en las mismas unidades que X (la varianza está en unidades²).
- **Regla 68-95-99.7:** $P(|X - \mu| \leq k\sigma) \approx 0.68, 0.95, 0.997$ para $k = 1, 2, 3$. Solo vale si se supone normalidad; sin suponerla, Chebyshev: $P(|X - \mu| \geq k\sigma) \leq 1/k^2$.

Ejemplo: Caramelos con $\mu = 5$, $\sigma = 0.05$; defectuoso fuera de $[4.95, 5.05] = \mu \pm 1\sigma \Rightarrow$ bajo normalidad $\approx 32\%$ defectuosos (mala calidad). **Ojo:** el enunciado dice "0.05 cm²": si fuera la varianza, $\sigma \approx 0.224$ y el porcentaje sería muchísimo mayor.

Ejemplo: Bayes con verosimilitudes continuas (tornillos): $P(B | x) = \frac{f_B(x)P(B)}{f_A(x)P(A) + f_B(x)P(B)}$. Con $x = 10.2$, $A \sim \mathcal{N}(10, 0.5^2)$, $B \sim \mathcal{N}(10.5, 0.7^2)$ y priors iguales (supuesto!): $f_A \approx 0.737$, $f_B \approx 0.520 \Rightarrow P(B | x) \approx 0.41$.

1.3 Máxima verosimilitud (MLE)

Bernoulli

$$k \text{ caras en } n \text{ tiros: } \mathcal{L}(p) = p^k(1-p)^{n-k}, \quad \ell(p) = k \log p + (n-k) \log(1-p), \quad \ell'(p) = 0 \Rightarrow \hat{p}_{MLE} = \frac{k}{n}$$

Pasos: (1) escribir la verosimilitud de los datos (i.i.d. $\Rightarrow$ producto), (2) tomar log (misma argmax, suma en vez de producto), (3) derivar e igualar a 0, (4) verificar que es máximo ($\ell'' < 0$).

Ejemplo: HHHHTHTHHT ($k = 6$, $n = 10$). Moneda justa: $0.5^{10} \approx 9.77 \times 10^{-4}$. Cargada $p = 0.7$: $0.7^6 0.3^4 \approx 9.53 \times 10^{-4} \Rightarrow$ la justa es (apenas) más verosímil. MLE: $\hat{p} = 0.6$.

2 Introducción: tipos de problemas, hipótesis, sesgo inductivo

Guía 2

2.1 Tipos de problema

- **Supervisado:** hay etiqueta y (discurso de odio, fraude). **No supervisado:** sin etiqueta, buscar estructura (agrupar voces). Segmentación de imágenes: suele ser supervisado (máscaras anotadas) — justificar.
- **Clasificación:** y en un conjunto finito (bool, enum/categorico). **Regresión:** $y \in \mathbb{R}$ (float).
- Casos borde: *predecir una probabilidad* $\rightarrow$ si las etiquetas anotadas son de clase, es clasificación (el modelo devuelve $P(y | x)$); si se anotó un número en $[0, 1]$, regresión. Nota 0..10: ordinal — puede plantearse como clasificación multiclase o regresión (entero). “Dónde vive”: categórica (barrio) o regresión 2D (lat, long). Próxima palabra: clasificación multiclase con $|V|$ clases. Subconjunto de objetos: **multi-etiqueta** (vector de bool).
- Métrica y dataset: siempre justificar (p.ej. fraude $\rightarrow$ recall/F1/AUC-PR por desbalance, no accuracy).

2.2 Espacio de instancias y de hipótesis

Conteos

$$|X| = \prod_{j=1}^p |\text{Dom}(A_j)| \quad |H_{\text{todas}}| = |Y|^{|X|}$$

Ejemplo: Color (4) $\times$ Tamaño (2): $|X| = 8$, todas las funciones $X \rightarrow \{A, B\}$: $2^8 = 256$. Sesgo “Si Color=rojo entonces A si no B ” o su inversa: $|H| = 2$. Si permito “Color = c ” para cualquiera de los 4 colores y 2 orientaciones: $|H| = 8$.

Parámetros mínimos de una hipótesis: recta con orientación $w_1x_1 + w_2x_2 + b \geq 0$: 3 números pero hay 1 grado de libertad de escala $\Rightarrow$ 2 continuos + el signo/dirección. Dos rectas paralelas a un eje: 2 umbrales + 1 bit (horizontal/vertical). Elipse general: 5 parámetros (centro 2, semiejes 2, ángulo 1) $\Rightarrow$ 3 elipses = 15.

2.3 Aprendizaje de conceptos (Mitchell cap. 1–2)

- **Consistencia:** h consistente con D si $h(x) = c(x) \forall \langle x, c(x) \rangle \in D$.
- **Version space:** $VS_{H,D} = \{h \in H : \text{consistente}(h, D)\}$, representado por las fronteras S (más específicas) y G (más generales).
- **Find-S:** arranca en la hipótesis más específica y generaliza lo mínimo con cada positivo (ignora negativos). **Candidate-Elimination:** mantiene S y G usando positivos y negativos.
- **Hipótesis de aprendizaje inductivo:** una h que aproxima bien c en muchos datos de entrenamiento lo hará también en datos no vistos.

Sesgo inductivo

Conjunto mínimo de supuestos B tal que $(\forall x \in X) [(B \wedge D \wedge x) \vdash L(x, D)]$. Sin sesgo inductivo **no hay generalización**.

- **Sesgo de restricción (lenguaje):** se limita H (ej. solo rectas). **Sesgo de preferencia (búsqueda):** H completo pero se prefieren ciertas hipótesis (ej. árboles cortos, Occam).
- **Algoritmo para separar con una recta** (datos lin. separables): Perceptrón — $w \leftarrow w + \eta y_i x_i$ ante cada error; converge si hay separabilidad, pero no elige “la mejor” recta. “Mejor” respecto al *margen* $\Rightarrow$ SVM. Si no son separables: perceptrón no converge $\rightarrow$ minimizar una pérdida (logística, hinge) o margen blando.

2.4 Entropía e información

Sorpresa y entropía

$$I(x) = -\log_2 p(x) = \log_2 \frac{1}{p(x)} \quad H(X) = \sum_x p(x) \log_2 \frac{1}{p(x)} = \mathbb{E}[I(X)]$$

- $0 \leq H \leq \log_2 K$ (K resultados); máxima con uniforme; 0 si un resultado tiene prob. 1. Convención $0 \log 0 = 0$.
- Evento con $p \rightarrow 0$: sorpresa $\rightarrow \infty$, pero su aporte a la entropía $p \log \frac{1}{p} \rightarrow 0$.
- Binaria: $H(p) = -p \log_2 p - (1-p) \log_2 (1-p)$; $H(0.5) = 1$.

Ejemplo: Dado justo: sorpresa de un 5 = $\log_2 6 \approx 2.585$ bits = H . Dado cargado (0.5, 0.1, ..., 0.1): $H = 0.5 \cdot 1 + 5 \cdot 0.1 \cdot 3.32 \approx 2.16$ bits < 2.585 $\Rightarrow$ **menos** incertidumbre.

Ejemplo: Región: 12 puntos (5A,7B): $H \approx 0.980$. Corte $x_1 < 0$: izq. (3A,2B) $H = 0.971$, der. (2A,5B) $H = 0.863$; ponderada $\frac{5}{12} 0.971 + \frac{7}{12} 0.863 \approx 0.908 \Rightarrow$ ganancia 0.072. Corte $x_2 < 0$: (3,3) $H = 1$, (2,4) $H = 0.918 \Rightarrow$ ponderada 0.959, ganancia 0.021 $\Rightarrow$ peor corte.

3 Árboles de decisión

Guía 3

3.1 Medidas de impureza y ganancia

Impureza de un nodo S con proporciones p_k

Entropía: $H(S) = -\sum_k p_k \log_2 p_k$

Gini: $G(S) = 1 - \sum_k p_k^2 = \sum_k p_k(1-p_k)$ (binaria: $2p(1-p)$, máx. 0.5)

Error de clasificación: $E(S) = 1 - \max_k p_k$

$$\Delta M(S, A) = M(S) - \sum_v \frac{|S_v|}{|S|} M(S_v) \quad (\text{Ganancia de información si } M = H; \text{ ganancia Gini si } M = G)$$

Ejemplo: Tabla "Salgo a caminar" (9 Sí / 5 No): $H(S) = 0.940$. Ganancias en la raíz: Cielo 0.246, Humedad 0.151, Viento 0.048, Temperatura 0.029 $\Rightarrow$ raíz = **Cielo**. Nublado es hoja pura (Sí). Rama Sol (2Sí/3No): mejor Humedad (Alta $\rightarrow$ No, Normal $\rightarrow$ Sí, ganancia 0.971). Rama Lluvia (3Sí/2No): mejor Viento (Débil $\rightarrow$ Sí, Fuerte $\rightarrow$ No).

3.2 Algoritmo (ID3 / CART)

- **Top-down, greedy (hill-climbing), sin backtracking:** en cada nodo elige el par $\langle a, c \rangle$ que maximiza ΔM ; recursión en los hijos. **No** explora todos los árboles (problema NP-hard) $\Rightarrow$ óptimo local.
- **Criterios de detención:** nodo puro, sin atributos, max_depth, min_samples_split, min_samples_leaf, ganancia mínima. Hoja $\rightarrow$ clase mayoritaria.
- **Atributos continuos:** ordenar valores únicos $v_1 < \dots < v_m$; candidatos = puntos medios $\frac{v_i + v_{i+1}}{2}$ (solo $m-1$; cortes entre dos valores consecutivos generan las mismas regiones, no repetirlos). Optimización: solo hace falta evaluar entre valores donde *cambia la clase*.
- **Probabilidades:** $\hat{P}(Y = k \mid x) =$ proporción de la clase k entre las instancias de entrenamiento de la hoja donde cae x .

mejor_corte($S, A, \Delta M$) – pseudocódigo

```
best=None; g*=-inf
for a in A:
    vals = sorted(set(S[a]))
    for i in range(len(vals)-1):
        c = (vals[i]+vals[i+1])/2; g = ΔM(S, (a,c))
        if g > g*: g*, best = g, (a,c)
return best, g*
```

3.3 Importancia de atributos

Disminución media de la impureza (MDI)

$$\text{Imp}(a) = \sum_{t: \text{nodo que corta por } a} \frac{N_t}{N} \Delta M(t), \quad \text{luego normalizar: } \frac{\text{Imp}(a)}{\sum_{a'} \text{Imp}(a')} \in [0, 1].$$

Para calcularla en el algoritmo: acumular en un diccionario $\text{imp}[a] += N_t/N * g$ cada vez que se elige un corte, y normalizar al final.

Importancia por permutación (modelo genérico M)

1) Medir la métrica en validación: s_0 . 2) Para cada atributo j : permutar al azar la columna j (rompe su relación con y conservando su distribución), re-evaluar $\rightarrow s_j$ (repetir varias veces y promediar). 3) $\text{Imp}(j) = s_0 - s_j$. No requiere reentrenar.

Ojo: Atributos correlacionados: en MDI el árbol elige uno de ellos y el otro queda con importancia baja (o se reparten arbitrariamente). En permutación, al permutar uno el modelo “usa” el otro $\Rightarrow$ ambos aparecen *menos* importantes de lo que son. MDI además está sesgado a favor de atributos de alta cardinalidad/continuos y se calcula sobre train.

3.4 Propiedades y fronteras

- Fronteras = **rectángulos alineados a los ejes** (cortes perpendiculares). Un árbol de altura h tiene a lo sumo 2^h hojas/regiones. Para decidir si una partición la genera un árbol: debe poder obtenerse con cortes recursivos que atraviesan *toda* la región actual (partición jerárquica).
- **Invariantes a transformaciones monótonas** de cada atributo ($m^2 \rightarrow km^2$, $\log$): mismo árbol, umbrales reescalados $\Rightarrow$ no hace falta escalar.
- Nuevo atributo $x_3 = x + y$: cortes $x + y \leq c \Rightarrow$ fronteras **oblicuas** (diagonales a -45°) en el plano original. Para graficar: armar grilla en (x, y) , calcular x_3 para cada punto, predecir y colorear.
- **Alta varianza / inestables:** cambiar pocas etiquetas (ruido 10%) puede cambiar el corte de la raíz y todo el árbol.
- **Altura real acotada:** árbol balanceado con hojas de ≥ 1 instancia $\Rightarrow$ altura $\leq \lceil \log_2 n \rceil$. Con $n = 1000$ no tiene sentido probar $\text{max_depth} \gg 10$. Además depende de: nodos puros antes, min_samples_leaf , instancias duplicadas/contradictorias, cantidad de valores distintos.

V/F clásicos: (a) “el objetivo es el árbol mínimo con hojas puras” $\rightarrow$ F (el objetivo es generalizar; hojas puras sobreajustan). (b) “explora todos los árboles” $\rightarrow$ F (greedy). (c) “un atributo aparece una sola vez por rama” $\rightarrow$ F para continuos (puede re-cortarse con otro umbral); V en ID3 con categóricos. (d) “un par $\langle a, c \rangle$ aparece una sola vez por rama” $\rightarrow$ V (repetirlo no divide nada). (e) “siempre 100% en train” $\rightarrow$ F; (f) solo si no hay instancias idénticas con etiquetas distintas $\rightarrow$ V.

Sesgo inductivo de ID3: preferencia (no restricción) por árboles **cortos** y con atributos de **mayor ganancia cerca de la raíz**. Desempates: la regla elegida (primer atributo, azar) *también* es parte del sesgo.

Poda: pre-poda (detener antes) o post-poda (*reduced-error pruning* con validación; *rule post-pruning*).

4 Selección de modelos, validación cruzada y Control

Guía 4

4.1 Esquema Desarrollo / Control



- **Control** se separa *al principio*, se usa **una sola vez** al final para estimar performance en datos nuevos. Si se usa para decidir, deja de ser insesgado.

- **K-fold CV**: partir Desarrollo en K folds disjuntos; entrenar K modelos, cada uno con $K - 1$ folds, validar en el restante; reportar media (y desvío). Los *conjuntos de validación* son disjuntos; los de *entrenamiento* se solapan.
- Variantes: **estratificado** (mantiene proporción de clases), **LOO** ($K = N$: poco sesgo pero alta varianza y costoso; los modelos *no* son independientes, comparten casi todos los datos), **GroupKFold** (un grupo entero en un solo fold: pacientes, hablantes, usuarios), **temporal** (entrenar en pasado, validar en futuro).
- **No partir al azar** cuando: hay grupos (mismo paciente/hablante en train y val $\Rightarrow$ optimista), series temporales (no ver el futuro), clases muy desbalanceadas (estratificar), duplicados o datos de la misma fuente/sesión.
- Tras elegir la configuración: **reentrenar con todo Desarrollo** y evaluar una vez en Control. No incluir Control en la CV.

Pseudocódigo

```
GRID_SEARCH(X, D, M): best=(None,None,-inf)
for (A,HS) in X: s = CROSS_VAL(A,HS,D,M); if s>best[2]: best=(A,HS,s)
return best

CROSS_VAL(A,HS,D,M,K=5): folds = partir(shuffle(D), K) (estratificado)
for k in 1..K: m = entrenar(A,HS, D\folds[k]); s[k]=M(m, folds[k])
return mean(s)

RANDOM_SEARCH(X,D,M,N): para cada (A,HS), repetir N veces: hs={k:SAMPLE(v) for k,v in HS},
evaluar con CROSS_VAL, quedarse con el mejor.

GROUP_K_CROSS_VAL: particionar los grupos (no las instancias) en K folds; foldk = instancias cuyos grupos
cayeron en k. Si hay más folds que grupos  $\Rightarrow K \leq |G|$  (usar leave-one-group-out). Si un grupo tiene el 60%:
folds muy desbalanceados; en esa iteración se entrena con solo el 40% y la estimación tiene alta varianza.
```

4.2 Optimismo y fugas de información (leakage)

- **Elegir el máximo entre miles de configuraciones** sobre los mismos folds es optimista (“maldición del ganador”: suerte en esos datos/semillas; usamos la validación para decidir) $\Rightarrow$ evaluar en Control. Respuestas correctas en 4.5: (b), (d), (e).
- CV **no evita** el overfitting; solo lo *estima/detecta* mejor.
- Control **no** es siempre peor que Desarrollo (puede ser mejor por azar), pero *se espera* que sea algo peor.
- **Todo lo que “aprende” de los datos** (medias para imputar, escaladores, selección de atributos por correlación con y , oversampling, target encoding) se ajusta **solo con el train de cada fold** y se aplica a val. Si no $\Rightarrow$ subestimación del error.
- **Oversampling**: la opción segura es (III): dentro de cada iteración, solo a los folds de entrenamiento. Antes de partir: copias casi idénticas en train y val/control $\Rightarrow$ performance sobreestimada. En validación: distorsiona la distribución real $\Rightarrow$ métrica no representativa.

Orden del proceso (4.7): definir métrica y mirar datos/distribución de etiquetas (g, h, l) $\rightarrow$ partir Desarrollo-Control (b) $\rightarrow$ baseline (n) $\rightarrow$ [loop] partir en folds (d); dentro de cada fold: imputar con media del train (m), seleccionar top-10 atributos con train (e), entrenar; grilla (j), análisis de errores (f), agregar configs (k) $\rightarrow$ entrenar modelo final con todo Desarrollo y exportar (c) $\rightarrow$ evaluar **una vez** en Control $\rightarrow$ reporte (a). Empezar de nuevo (i) tras mirar Control “quemar” el Control.

4.3 Métrica global vs. promedio de folds

Métricas aditivas

$$M(D) = \frac{1}{N} \sum_i \ell_i = \sum_{k=1}^K \frac{n_k}{N} M_k \quad (\text{Accuracy, MSE}). \text{ Si } n_k = N/K = \frac{1}{K} \sum_k M_k \text{ (promedio simple).}$$

Con folds de distinto tamaño hay que **ponderar por** n_k/N . RMSE no es aditiva: por Jensen ($\sqrt{\cdot}$ cóncava) $\frac{1}{K} \sum_k \sqrt{MSE_k} \leq \sqrt{\frac{1}{K} \sum_k MSE_k} \Rightarrow$ promedio de RMSE locales $\leq$ RMSE global. Contraejemplo: fold 1 errores 0, 0 (RMSE 0), fold 2 errores 2, 2 (RMSE 2); promedio 1, global $\sqrt{2} \approx 1.41$.

5.1 Clasificador óptimo de Bayes

Regla de Bayes

$$\hat{y}(x) = \arg \max_c P(Y = c | X = x) = \arg \max_c \frac{P(X = x | Y = c)P(Y = c)}{P(X = x)}$$

$$\hat{y}(x) = \arg \max_c P(X = x | Y = c) P(Y = c)$$

- Minimiza el error esperado (0-1); su error es el **error de Bayes** (irreducible). En la práctica no se construye porque **no conocemos** $P(X | Y)$ ni $P(Y)$: hay que estimarlas (y en alta dimensión es imposible sin supuestos).
- **Qué alcanza para decidir** (binario): $P(Y = 1 | x)$ sola ✓ (la otra es 1-ella). $P(x | Y = 1)$ y $P(x | Y = 0)$ sin priors × (salvo priors iguales). $P(x | Y = 1)$ y $P(Y = 0) \times$ (falta $P(x | Y = 0)$). $P(x)$ sola, o solo priors × (los priors solo dan la clase mayoritaria, ignorando x).
- **Frontera en una grilla**: para cada punto x de la grilla calcular $P(x | c)P(c)$ para cada clase, asignar el argmax, colorear; la frontera es donde cambia el argmax.

Ejemplo: Verde $\sim \mathcal{N}(0, 2^2)$, $\pi_V = 0.2$; Azul $\sim \mathcal{N}(3, 1)$, $\pi_A = 0.8$; $x = 2$: $f_V(2)\pi_V = 0.121 \cdot 0.2 = 0.024$; $f_A(2)\pi_A = 0.242 \cdot 0.8 = 0.194 \Rightarrow$ **Azul**.

5.2 Generativos vs. discriminativos

	Generativo	Discriminativo
Modela	$P(X Y)$ y $P(Y)$ (cómo se generan los datos)	$P(Y X)$ o directamente la frontera
Ejemplos	Naive Bayes, LDA, QDA	Regresión logística, SVM, árboles, KNN
Pros	pocos datos, puede generar muestras, maneja faltantes	suele ser más preciso con muchos datos, menos supuestos

5.3 KNN

- Predice la clase mayoritaria (o promedio en regresión) de los K vecinos más cercanos según una distancia (euclídea $\sqrt{\sum_j (x_j - x'_j)^2}$, Manhattan, etc.). No paramétrico, *lazy* (no entrena).
- $K = 1$: frontera muy irregular (celdas de Voronoi), 100% en train, **overfitting**/alta varianza. $K = n$: predice siempre la clase mayoritaria (frontera inexistente: todo el plano de una clase), **underfitting**/alto sesgo.
- **Escalar es obligatorio**: altura en metros (~ 1.5 - 2.1) vs. edad en años (~ 0 - 90) $\Rightarrow$ la edad domina la distancia. Estandarizar o min-max. Categóricas $\rightarrow$ one-hot.

Maldición de la dimensión

Fracción usada con un 10% por eje: 0.1^p ($p = 1$: 10%, $p = 2$: 1%, $p = 100$: $10^{-98}\%$). Lado del hipercubo que contiene el 10% de los datos: $\ell = 0.1^{1/p}$ ($p = 1$: 0.1; $p = 2$: 0.316; $p = 100$: 0.977) $\Rightarrow$ en alta dimensión los "vecinos" ya no son cercanos.

5.4 LDA y QDA

Supuestos LDA: $X | Y = c \sim \mathcal{N}(\mu_c, \Sigma)$ con **covarianza común** a todas las clases; priors π_c .

Función discriminante ($p = 1$)

$$\delta_c(x) = x \frac{\mu_c}{\sigma^2} - \frac{\mu_c^2}{2\sigma^2} + \log \pi_c$$

Multivariada: $\delta_c(x) = x^\top \Sigma^{-1} \mu_c - \frac{1}{2} \mu_c^\top \Sigma^{-1} \mu_c + \log \pi_c$

Derivación: $\arg \max_c P(c | x) = \arg \max_c \pi_c f_c(x) = \arg \max_c \log \pi_c + \log f_c(x)$. Con $\log f_c(x) = -\log(\sigma\sqrt{2\pi}) - \frac{x^2}{2\sigma^2} + \frac{x\mu_c}{\sigma^2} - \frac{\mu_c^2}{2\sigma^2}$, los términos $-\log(\sigma\sqrt{2\pi})$ y $-\frac{x^2}{2\sigma^2}$ **no dependen de c** (porque σ es común) y se descartan.

Frontera $k = 2$: $\delta_1 = \delta_2 \Rightarrow x^* = \frac{\mu_1 + \mu_2}{2} + \frac{\sigma^2}{\mu_1 - \mu_2} \log \frac{\pi_2}{\pi_1}$; si $\pi_1 = \pi_2$: $x^* = \frac{\mu_1 + \mu_2}{2}$. Un prior mayor *corre la frontera hacia la otra clase* (agrandando su región).

QDA: Σ_c distinta por clase $\Rightarrow$ el término $-\frac{x^2}{2\sigma_c^2}$ ya no se cancela: $\delta_c(x) = -\frac{1}{2} \log |\Sigma_c| - \frac{1}{2}(x - \mu_c)^\top \Sigma_c^{-1}(x - \mu_c) + \log \pi_c \Rightarrow$ frontera **cuadrática**.

5.5 Naive Bayes

Gaussian NB

$$\hat{y} = \arg \max_c P(Y = c) \prod_{j=1}^p P(X_j = x_j | Y = c) \approx \arg \max_c \hat{\pi}_c \prod_{j=1}^p f_{\mathcal{N}}(x_j; \hat{\mu}_{c,j}, \hat{\sigma}_j)$$

Sesgo inductivo (los dos $\approx$): (1) **independencia condicional** de los atributos dada la clase; (2) cada $X_j | Y$ es **normal**, con media por clase y atributo $\hat{\mu}_{c,j}$ y (en la fórmula de la guía) desvío $\hat{\sigma}_j$ que depende solo del atributo, *compartido entre clases*. También: los priors se estiman con frecuencias. Categóricos: $P(x_j | c) = \frac{\#(x_j, c) + 1}{\#c + |V_j|}$ (suavizado de Laplace). En la práctica: usar sumas de logs.

5.6 Regresión logística

Modelo

$$P(Y = 1 | x) = \sigma(w^\top x + b) = \frac{1}{1 + e^{-(w^\top x + b)}}, \quad \log \frac{p}{1-p} = w^\top x + b \text{ (log-odds lineal)}$$

Pérdida (log-loss / cross-entropy): $\mathcal{L} = -\frac{1}{N} \sum_i [y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i)]$, se minimiza con descenso por gradiente ($\nabla = \frac{1}{N} \sum_i (\hat{p}_i - y_i) x_i$).

Frontera **lineal** ($w^\top x + b = 0$ con umbral 0.5). Discriminativo; salida probabilística; con regularización L_2 (C en sklearn = $1/\lambda$).

5.7 SVM

Margen máximo

Hard margin: $\min_{w,b} \frac{1}{2} \|w\|^2$ s.a. $y_i(w^\top x_i + b) \geq 1$ (margen = $2/\|w\|$).

Soft margin: $\min \frac{1}{2} \|w\|^2 + C \sum_i \xi_i$, $y_i(w^\top x_i + b) \geq 1 - \xi_i$, $\xi_i \geq 0$ (hinge: $\max(0, 1 - y_i f(x_i))$).

- **Vectores de soporte:** puntos sobre el margen o dentro; solo ellos definen la frontera.
- C grande: poco error de train, margen chico (varianza $\uparrow$). C chico: margen ancho, más errores (sesgo $\uparrow$).
- **Kernel trick:** reemplazar $x^\top x'$ por $K(x, x') = \phi(x)^\top \phi(x') \Rightarrow$ separador lineal en un espacio transformado. Polinomial $(x^\top x' + c)^d$; RBF $\exp(-\gamma \|x - x'\|^2)$ (γ grande $\Rightarrow$ frontera más curva).
- Clases concéntricas (círculo dentro de anillo): ninguna recta las separa; con $\phi(x) = (x_1, x_2, x_1^2 + x_2^2)$ o kernel RBF/polínomial de grado 2 se separan con un plano.

5.8 Fronteras de decisión típicas

Overfitting	frontera muy irregular que rodea puntos individuales / ruido
Underfitting	frontera demasiado simple (ej. recta para datos curvos, o todo una clase)
Árbol altura 3	≤ 8 rectángulos alineados a los ejes, cortes jerárquicos
KNN $K = 1 / K = n$	mosaico de Voronoi / todo el plano de la clase mayoritaria
LDA / Logística / SVM lineal	una recta (hiperplano); en LDA desplazada hacia la clase de menor prior
QDA	cónicas (elipses, parábolas, hipérbolas)

6 Métricas

Guía 6

6.1 Regresión

$$MAE = \frac{1}{n} \sum_i |y_i - \hat{y}_i| \quad MSE = \frac{1}{n} \sum_i (y_i - \hat{y}_i)^2 \quad RMSE = \sqrt{MSE}$$

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2} \quad MAPE = \frac{100\%}{n} \sum_i \frac{|y_i - \hat{y}_i|}{|y_i|}$$

- Transformación $y \rightarrow ay + b$, $\hat{y} \rightarrow a\hat{y} + b$ ($a > 0$): $MAE \rightarrow a MAE$, $MSE \rightarrow a^2 MSE$ (b se cancela), R^2 invariante.
- $R^2 = 1$: predicción perfecta. $R^2 = 0$: tan bueno como predecir siempre $\bar{y}$. $R^2 < 0$: peor que predecir la media. Ventaja: adimensional, comparable entre problemas con distintas unidades. Desventaja: no dice el tamaño del error en unidades reales, depende de la varianza de y en el dataset.
- MSE penaliza más los errores grandes (sensible a outliers); MAE más robusto.
- MAPE: se indefinir/explota con $y_i \approx 0$. Asimétrico con $y = 100$: predecir 0 $\rightarrow 100\%$; 200 $\rightarrow 100\%$; 300 $\rightarrow 200\%$. Subestimar está acotado a 100%, sobreestimar no $\Rightarrow$ empuja a **subestimar**.

6.2 Matriz de confusión y métricas de umbral fijo

	Pred. +	Pred. -
Real +	TP	FN
Real -	FP	TN

Clase **positiva**: la que queremos detectar / la de interés (spam, fraude, enfermo). Perros vs. gatos: es arbitrario $\Rightarrow$ la métrica cambia según cuál se elija.

Accuracy	$\frac{TP + TN}{N}$	Precision	$\frac{TP}{TP + FP}$
Recall = TPR = sensibilidad	$\frac{TP}{TP + FN}$	Especificidad = TNR	$\frac{TN}{TN + FP}$
FPR	$\frac{FP}{FP + TN} = 1 - TNR$	Balanced acc.	$\frac{1}{2}(TPR + TNR)$
$F_\beta = (1 + \beta^2) \frac{P \cdot R}{\beta^2 P + R} = \frac{(1 + \beta^2) TP}{(1 + \beta^2) TP + \beta^2 FN + FP}$		$F_1 = \frac{2TP}{2TP + FN + FP}$	

Demostración F_β : sustituir $P = \frac{TP}{TP + FP}$, $R = \frac{TP}{TP + FN}$; numerador $(1 + \beta^2) \frac{TP^2}{(TP + FP)(TP + FN)}$, denominador $\frac{\beta^2 TP(TP + FN) + TP(TP + FP)}{(TP + FP)(TP + FN)}$; simplificar un TP . $\beta > 1$ favorece recall, $\beta < 1$ favorece precision.

- TN no aparece** en F_β , precision ni recall: agregar 90.000 legítimos bien clasificados no cambia $F_1 \Rightarrow$ no captura la capacidad de descartar negativos; comparar entre poblaciones con distinta prevalencia es engañoso.

- **Accuracy con desbalance:** 99% negativos $\Rightarrow$ clasificador constante “-” logra 99% y no detecta nada.
- **Balanced accuracy** $= \frac{1}{N} \left[\frac{TP}{2p} + \frac{TN}{2(1-p)} \right]$ con $Np = TP + FN \Rightarrow \frac{1}{2}(TPR + TNR) \in [0, 1]$. Clasificador constante (cualquier clase) $\Rightarrow 0.5$.
- **FN más grave que FP:** diagnóstico de cáncer, fraude. **FP más grave:** spam (perder un mail real) $\Rightarrow$ priorizar **precision** o F_β con $\beta < 1$ (este último es mejor: no ignora totalmente el recall). AUC-ROC no fija umbral, no sirve para esto.

6.3 Umbral, ROC y PR

Score $s(x)$ (probabilidad, distancia al hiperplano, logit); predecir + si $s(x) \geq \theta$. Cada umbral $\rightarrow$ una matriz de confusión; hay **finitas** matrices distintas (a lo sumo $N + 1$), no infinitas.

- **Recall es no creciente en θ :** si $\theta_1 > \theta_2$, $\{s \geq \theta_1\} \subseteq \{s \geq \theta_2\} \Rightarrow TP(\theta_1) \leq TP(\theta_2)$ y el denominador $TP + FN = \#positivos$ es constante.
- **Precision no es monótona:** scores ordenados (mayor a menor) con etiquetas +, -, +. θ alto (solo el 1ro): $P = 1$; bajando (2 primeros): $P = 0.5$; bajando (3): $P = 0.67$. Subir el umbral *no siempre* sube la precision.
- **ROC:** TPR vs. FPR para todos los umbrales. **PR:** precision vs. recall. La ROC **no usa precision**.
- **AUC-ROC** $= P(s(x^+) > s(x^-))$: probabilidad de rankear bien un par positivo-negativo. 0.5 = azar, 1 = perfecto. Invariante al umbral y a **cualquier transformación monótona creciente** de los scores (solo importa el orden) $\Rightarrow$ **no distingue** un modelo confiado de uno poco confiado con el mismo orden (no mide calibración).
- Con desbalance fuerte, AUC-PR es más informativa que AUC-ROC.
- F_1 de dos modelos se cruza al cambiar umbral: F_1 depende del umbral y de la calibración de los scores; un modelo puede tener scores “corridos”. Con dos puntos no podemos concluir nada sobre el AUC-PR (resume todos los umbrales).

CURVA-ROC(labels, scores)

```
P = #True; N = #False; tp = fp = 0; out = [(+inf, 0, 0)]
for (s, y) in sorted(zip(scores, labels), desc):
    if y: tp += 1 else: fp += 1
    if es el último con este score: out.append((s, fp/N, tp/P)) (empates juntos)
return out    AUC = área por trapecios.
```

6.4 Multiclase y costos

- average en sklearn: **binary** (solo la clase positiva), **macro** (promedio simple por clase, todas pesan igual), **weighted** (ponderado por soporte), **micro** (suma global de TP/FP/FN; en multiclase single-label = accuracy).
- F_1 (gato) alto no implica mejor clasificador: ignora cómo se clasifican los perros (TN). Mirar ambas clases o macro- F_1 .

Decisión de costo mínimo

$$d^*(x) = \arg \min_d \sum_y \hat{P}(y | x) C(y, d) \quad (\text{requiere probabilidades calibradas})$$

Ejemplo: $p = P(\text{Maligno})$. Costos esperados: Ignorar $= 100p$; ReChequear $= 5p + 15(1 - p)$; Operar $= 50(1 - p)$. $p = 0.3$: 30 / **12** / 35 $\rightarrow$ ReChequear. $p = 0.99$: 99 / 5.1 / **0.5** $\rightarrow$ Operar. $p = 0.01$: **1** / 14.9 / 49.5 $\rightarrow$ Ignorar.

Ojo: “La matriz de confusión se pondera por la probabilidad” $\rightarrow$ F (se construye con predicciones duras tras un umbral). “Sin FP $\Rightarrow$ precision $= 1$ ” $\rightarrow$ V (si hay al menos un TP; si no, indefinida). “Sin FN $\Rightarrow$ recall $= 1$ ” $\rightarrow$ V. “Precision y recall ignoran los negativos” $\rightarrow$ V en el sentido de que no usan TN (precision sí usa FP).

7 Sesgo y varianza

Guía 7

Descomposición (error cuadrático)

Supuestos: $y = f(x) + \varepsilon$, $\mathbb{E}[\varepsilon] = 0$, $\text{Var}(\varepsilon) = \sigma^2$, ε independiente del dataset D_n . Sea $\bar{h}(x) = \mathbb{E}_{D_n}[\hat{h}_{D_n}(x)]$.

$$\mathbb{E}_{D_n, \varepsilon} [(y - \hat{h}_{D_n}(x))^2] = \underbrace{(f(x) - \bar{h}(x))^2}_{\text{Sesgo}^2} + \underbrace{\mathbb{E}[(\hat{h}_{D_n}(x) - \bar{h}(x))^2]}_{\text{Varianza}} + \underbrace{\sigma^2}_{\text{ruido irreducible}}$$

Demostración: escribir $y - \hat{h} = (f - \bar{h}) + (\bar{h} - \hat{h}) + \varepsilon$, elevar al cuadrado y tomar esperanza. Los productos cruzados se anulan: $\mathbb{E}[\bar{h} - \hat{h}] = 0$, $\mathbb{E}[\varepsilon] = 0$, e independencia de ε y $\hat{h}$.

Con error = $y - \text{pred}$ (sin cuadrado): $\mathbb{E}[y - \hat{h}] = f - \bar{h}$ = solo el sesgo; los errores positivos y negativos se cancelan, no aparece la varianza ni el ruido $\Rightarrow$ no sirve como medida de error.

	Alto sesgo (underfitting)	Alta varianza (overfitting)
Síntoma	error de train alto (y val alto)	train muy bueno, gran brecha con val
Causa	modelo demasiado simple	modelo demasiado complejo / pocos datos
Remedios	modelo más complejo, más atributos, menos regularización	más datos , regularizar, podar, ensambles (bagging), menos atributos

Ejemplo: Tabla 7.3 (train/val): C1 .99/.89 alta varianza (overfit); C2 .90/.89 sesgo alto relativo (si otro modelo llega a .98); C3 .90/.75 sesgo alto y varianza alta; C4 .99/.98 el mejor. Si hasta un humano falla mucho (error de Bayes alto), C2 podría estar cerca del óptimo $\Rightarrow$ **no** diríamos sesgo alto: el sesgo se mide contra el error alcanzable. Si C1 y C2 son árboles: C1 profundo, C2 poco profundo.

V/F: más datos ayuda a la varianza (V), no al sesgo (F). Modelo complejo $\rightarrow$ varianza alta (V), sesgo alto (F). Sesgo alto $\leftrightarrow$ underfitting (V); varianza alta $\leftrightarrow$ overfitting (V, en general, aunque no son sinónimos exactos: sobreajuste es la brecha train-val observada; varianza es sensibilidad al dataset).

8 Ensamblés

Guía 8

8.1 Bagging

- **Bootstrap:** B muestras de tamaño n **con reposición**; entrenar un modelo en cada una (típicamente árboles profundos: bajo sesgo, alta varianza); agregar por **votación** (clasificación) o **promedio** (regresión).
- Probabilidad de que una instancia *no* esté en un bootstrap: $(1 - \frac{1}{n})^n \rightarrow e^{-1} \approx 0.368 \Rightarrow$ cada modelo ve $\approx 63.2\%$ de instancias distintas.
- Reduce **varianza** sin aumentar mucho el sesgo.

Varianza de un promedio

B modelos con varianza σ^2 y correlación ρ : $\text{Var}(\frac{1}{B} \sum_b h_b) = \rho \sigma^2 + \frac{1-\rho}{B} \sigma^2 \Rightarrow$ bajar ρ reduce el piso de varianza.

- **OOB (out-of-bag):** para cada instancia, predecir solo con los modelos que no la vieron ($\approx B/3$) y calcular el error $\Rightarrow$ estimación de generalización sin CV. Tiende a ser algo **pesimista** (sobreestima el error) porque cada predicción usa menos modelos que el ensamble completo.
- Si los submodelos votan duro y el ensamble reporta la fracción de votos: probabilidades $[0.75, 0.5, 0.25, 0.75, 1]$ son múltiplos de $1/4 \Rightarrow B$ es múltiplo de 4 (mínimo 4; no se puede determinar exactamente).

8.2 Random Forest (Breiman 2001)

- Bagging de árboles + en **cada nodo/corte** se eligen al azar m de los p atributos candidatos (típico $m = \sqrt{p}$ clasif., $p/3$ regresión) $\Rightarrow$ árboles **decorrelacionados** $\Rightarrow$ más reducción de varianza que Bagging. $m = p \Rightarrow$ Bagging.
- $m = 1$: en cada nodo se usa un atributo elegido al azar (sin elegir el mejor atributo, solo el mejor corte); el árbol **puede** ser profundo y usar muchos atributos.
- **Importancia (Breiman)**: por permutación en OOB: para cada árbol, permutar el atributo j en su OOB y medir el aumento de error; promediar. **sklearn** (`feature_importances_`): MDI — suma de la ganancia ponderada en cada corte por j , promediada entre árboles ($\div B$) y normalizada. Existe `permutation_importance` aparte.

V/F: bootstrap tiene tamaño n (V, en Bagging y RF). “En RF cada árbol usa un subconjunto de atributos” $\rightarrow$ F: el sorteo es **por corte**, no por árbol. $m = 1 \Rightarrow$ “un nivel” o “un atributo” $\rightarrow$ F ambas. “Bagging reduce más varianza que RF” $\rightarrow$ F. Importancia como suma de ganancias (f) y dividido B (g): ambas son proporcionales (mismo ranking); sklearn usa el promedio normalizado. OOB estima error sin CV y tiende a **sobreestimar**lo (j) más que subestimar.

8.3 Boosting

Weak learner: apenas mejor que el azar (error < 0.5), ej. *stumps* (árboles de 1 corte). Boosting reduce principalmente el **sesgo**; los modelos se entrenan **secuencialmente**.

AdaBoost ($y \in \{-1, +1\}$)

Inicio $w_i = 1/n$. Para $t = 1..T$: entrenar h_t con pesos w ; $\varepsilon_t = \sum_i w_i \mathbb{I}[h_t(x_i) \neq y_i]$; $\alpha_t = \frac{1}{2} \ln \frac{1-\varepsilon_t}{\varepsilon_t}$;
 $w_i \leftarrow w_i e^{-\alpha_t y_i h_t(x_i)}$ (sube si se equivocó, baja si acertó), normalizar. Final: $H(x) = \text{sign}(\sum_t \alpha_t h_t(x))$.

Cantidad de modelos T : hiperparámetro, elegido por validación cruzada / early stopping (o frenar si $\varepsilon_t \geq 0.5$ o $= 0$).

Gradient Boosting: cada nuevo modelo ajusta el **gradiente negativo** de la pérdida (residuos $y - \hat{y}$ en MSE): $F_t = F_{t-1} + \eta h_t$, con learning rate η . AdaBoost = caso particular con pérdida exponencial que *reponde instancias*; GB *ajusta residuos*. **XGBoost**: usa gradiente y hessiano (2do orden), regularización L_1/L_2 sobre hojas y penalización por #hojas, subsampling de filas/columnas, manejo nativo de faltantes.

9 Ingeniería de atributos

Guía 9

Escalado

$$\text{Min-max: } x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \in [0, 1] \quad \text{Z-score: } x' = \frac{x - \mu}{\sigma} \quad \text{Robust: } x' = \frac{x - \text{mediana}}{IQR}$$

- Min-max con casi todo en 0 y un valor en 1 $\Rightarrow$ **outlier** (o distribución muy asimétrica: precios). Alternativas: estandarizar si es normal; robust scaler o $\log(1 + x)$ si hay outliers/cola larga; recortar (clipping).
- Sin outliers y no normales $\Rightarrow$ min-max es razonable.
- Todos los parámetros ($\mu, \sigma, \min, \max$, media para imputar) se calculan **solo con train** (en cada fold) y se aplican a val/control/producción.

9.1 Valores faltantes

- Imputar con la media de *todo* el dataset (incluyendo val/control) $\Rightarrow$ fuga de información $\Rightarrow$ **subestimación del error**. Correcto: calcular la media con el train de cada fold; para el modelo final, con todo Desarrollo; en **producción** usar ese mismo valor guardado (no la media de los datos nuevos).
- Estrategias: media/mediana/moda; **columna indicadora** “era faltante” (el faltante puede ser informativo); valor centinela (-1) — funciona bien con **árboles** (lo separan con un corte) pero mal con KNN/lineales; imputación por regresión/KNN a partir de otras columnas. Para árboles, (b), (c) y (d) son buenas; (d) es la más segura.

9.2 Categóricas

- **One-hot:** una columna binaria por valor (nominales: barrio, color). Dimensiones: Ciudad (1000) + Provincia (23) + Edad + Salario = 1025 (o 1023 si se descarta una columna por variable, `drop='first'`). Alta cardinalidad $\Rightarrow$ muchas dimensiones (maldición de la dimensión); alternativas: agrupar poco frecuentes, frequency/target encoding (este último con cuidado de leakage).
- **Ordinal:** si hay orden natural (chico < mediano < grande $\rightarrow$ 0, 1, 2).
- Ciudad y Provincia son redundantes (ciudad determina provincia).
- **Orden de pasos:** (b) imputar $\rightarrow$ (c) codificar categóricas $\rightarrow$ (a) escalar (escalar al final para que las columnas imputadas/nuevas queden en la misma escala; el imputado no debe distorsionar el min/max). No escalar las one-hot (ya están en $\{0, 1\}$).

9.3 Nuevos atributos (ejemplos para KNN)

- **Fechas/horas:** día de la semana, mes, feriado/fin de semana, “días desde publicación”, hora. Variables **cíclicas:** $(\sin \frac{2\pi h}{24}, \cos \frac{2\pi h}{24})$ para que 23h y 0h queden cerca. Fecha puntual relevante (año nuevo) $\Rightarrow$ flag o “días hasta año nuevo”.
- **Lat/long:** distancia a un punto de interés (barrio chino, estación de bicis, centro), en km; o cluster/barrio.
- Precio por m², m² por habitación; texto (nombre/apellido) $\rightarrow$ n-gramas de caracteres, longitud, terminaciones.
- Siempre: escalar numéricos, one-hot a nominales, imputar faltantes (sin leakage).
- **Árboles vs. KNN:** los árboles **no necesitan escalar** (invariantes a transformaciones monótonas) ni sufren atributos irrelevantes tanto; pueden usar atributos en unidades distintas tal cual. KNN sí necesita escalar y quitar atributos irrelevantes.

10 Chuleta final

- $H = -\sum p \log_2 p$; $G = 1 - \sum p^2$; ganancia = $M(S) - \sum \frac{|S_v|}{|S|} M(S_v)$.
- Bayes óptimo: $\arg \max_c P(x | c)P(c)$.
- LDA $p = 1$: $\delta_c = x \frac{\mu_c}{\sigma^2} - \frac{\mu_c^2}{2\sigma^2} + \log \pi_c$.
- Frontera LDA: $\frac{\mu_1 + \mu_2}{2} + \frac{\sigma^2}{\mu_1 - \mu_2} \log \frac{\pi_2}{\pi_1}$.
- NB: $\hat{\pi}_c \prod_j P(x_j | c)$ (indep. condicional).
- Logística: $\sigma(w^\top x + b)$, log-loss.
- SVM: margen $2/\|w\|$, C , kernels.
- KNN: escalar; $0.1^{1/p}$.
- $P = \frac{TP}{TP+FP}$, $R = \frac{TP}{TP+FN}$, $FPR = \frac{FP}{FP+TN}$.
- $F_\beta = \frac{(1+\beta^2)TP}{(1+\beta^2)TP+\beta^2FN+FP}$.
- $AUC = P(s^+ > s^-)$, invariante a monótonas.
- $R^2 = 1 - SS_{res}/SS_{tot}$; $MAE \times a$, $MSE \times a^2$.
- Error = Sesgo² + Var + σ^2 .
- Bootstrap: 63.2% únicos, 36.8% OOB.
- Var = $\rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$.
- AdaBoost: $\alpha = \frac{1}{2} \ln \frac{1-\epsilon}{\epsilon}$.
- Métrica global = $\sum_k \frac{n_k}{N} M_k$ (aditivas).
- MLE Bernoulli: $\hat{p} = k/n$.
- Todo “fit” de preprocesamiento: solo en train.